# Intermediate (make search conditional) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.


- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing


## Hint 1 — The idea, and the exact pieces

The exact pieces:

- A small graph with `task`, `sources`, `answer`; a `search_node` (Doc08's `retrieve()`) and an `answer_node` (answers from the sources if there are any, otherwise directly).
- `builder.add_conditional_edges(START, should_search, {"search": "search_node", "skip": "answer_node"})` — a conditional edge from `START` is the entry decision. `builder.set_conditional_entry_point(should_search, {...})` is a shortcut for the same thing.
- Pass the router into `build_graph(router)` as a parameter (Doc08's `chunk_fn` idea), so you can swap routers without rebuilding anything.

Then compare strategies: keywords are free but miss unexpected wording; a model call ("Answer with exactly one word: yes or no") catches more but costs a call every time. A **hybrid** uses keywords for clear cases, skips short chatty messages, and only asks the model about the ambiguous middle — and logs which branch fired and why.

## Hint 2 — The plan, and almost the whole thing

```
hybrid should_search(state):
    if has_keyword(task):                       -> "search"   (free)
    if len(task.split()) <= 6 and no "?":       -> "skip"     (free)
    otherwise ask_model_to_classify(task)       (one model call)

logging: get_logger("routing").info(f"routed=... reason=... task=...")
```

One piece to get you unstuck:

```
def has_keyword(task: str) -> bool:
    lowered = task.lower()
    for keyword in KEYWORDS:
        if keyword in lowered:
            return True
    return False
```

Full solution: Show me the solution